

Workshop Rapporter

ITF20123-1 25V - Rammeverk og .NET

Andreas Isaksen

10. februar 2025
Halden



Innhold

1	Workshop 1	1
1.1	Del 1	1
1.1.1	Gruppen vår	1
1.1.2	Oppsummering av koden	1
1.1.3	Hvordan fungerer koden?	1
1.1.4	Programflyt	2
1.1.5	Endring på uttrykk i kode	2
1.1.6	Klasser	3
1.1.7	Metoder	4
1.1.8	Variabler	4
1.1.9	Konstanter	5
1.2	Del 2	6
1.2.1	Gruppen vi møtte	6
1.2.2	Referat	6
2	Workshop 2	7
2.1	Gruppen vår	7
2.2	Analyse av koden	7
2.2.1	Library 1 (Andreas Isaksen, Andreas Olaussen)	7
2.2.1.1	Scenario 1 – Prisjakt komplett PC (Prissammenligning)	7
2.2.1.2	Scenario 2 – Lagerstyring for PC-leverandør	8
2.2.1.3	Scenario 3 – API løsning som bygger en PC basert på valgte komponenter innenfor et budsjett	9
2.2.1.4	Library 1 – Retningslinjer som biblioteket bryter	10
2.2.2	Library 2 (Erik Hankø, Abdelkader Adam Niane)	10
2.2.2.1	Scenario 1 – Bygg en PC med begrenset budsjett	10

2.2.2.2	Scenario 2 – Optimalisere ordre med flere butikker	11
2.2.2.3	Scenario 3 – Håndtering av ekvivalensgrupper	11
2.2.2.4	Library 2 – Retningslinjer som biblioteket bryter	12
2.3	Referat	13
3	Workshop 3	14
3.1	Gruppen vår	14
3.2	Organisering	14
3.3	Analyse av koden	15
3.4	Brukertest	15
3.4.1	S1-medlemmer observerer S2-medlemmer	15
3.4.1.1	Oppgave	15
3.4.1.2	Rollefordeling	16
3.4.1.3	Fremgangsmåte – Oppgave A	16
3.4.1.4	Fremgangsmåte – Oppgave B	17
3.4.1.5	Program kode skrevet av gruppe S2	17
3.4.2	S2-medlemmer observerer S1-medlemmer	18
3.4.2.1	Oppgave	18
3.4.2.2	Rollefordeling	18
3.4.2.3	Fremgangsmåte – Oppgave A	19
3.4.2.4	Fremgangsmåte – Oppgave B	19
3.4.2.5	Program kode skrevet av gruppe S1	19
3.5	Referat	20
3.6	Oppsummering	20
3.6.1	Oppgave 1 - Kode	20
3.6.1.1	Oppgave A	20
3.6.1.2	Oppgave B	20

3.6.2	Oppgave 2 - Kode	21
3.6.2.1	Oppgave A	21
3.6.2.2	Oppgave B	21

Workshop 1

1.1 Del 1

1.1.1 Gruppen vår

Etter at vi hadde blitt introdusert for oppgaven, ble vi delt inn i tilfeldige grupper til å utføre workshop. Min gruppe ble tildelt følgende gruppe nummer og bestod av disse deltagerne.

Gruppenummer:

- 14

Deltagere:

- Andreas Isaksen
- Markus Kristoffer Olsen
- Choju Jansen

1.1.2 Oppsummering av koden

Fra vår gruppe sin tolkning, har vi konkludert med at dette programmets hensikt er å lese en tekstfil (CSV format), som videre blir delt opp i stringer basert på definerte kriterier i programmet. Disse strengene blir videre indeksert, og til slutt itererer programmet gjennom denne indeksen og skriver resultatet ut i konsollen.

1.1.3 Hvordan fungerer koden?

- **prog.cs** - Denne filen kjører selve programmet og har ansvar for kodeflyt, samt utskrift til konsoll.
- **YEY!.cs** - Denne filen håndterer selve innholdet i tekstfilen. Det er her det blir utført kontroll på filens innhold og lokasjon, samt at logikk for hvordan teksten skal deles opp i strenger finnes her.
- **Litt_kode.cs** - Denne filen inneholder klasse objekter som holder på data som blir generert i *YEY!.cs*. Denne koden fungerer som et mellomledd for data som kommer fra *prog.cs* og blir behandlet i *YEY!.cs*.
Det er her datastrukturen som blir brukt i hele programmet defineres.

1.1.4 Programflyt

Dette er en beskrivelse av hvordan programmet forløper seg fra start til slutt.

1. **Programmet starter** - Selve programmet starter ved eksekveringen av *Main* metoden i filen *prog.cd*.
Her blir det opprettet en *string* variabel med navnet *path*, som holder på fil lokasjonen til tekstfilen.
2. **Deklarerer app objekt** - På neste linje i *Main* metoden blir det deklart et nytt objekt av *app* klassen under instans variabelen med navnet "a". Det er denne variabelen som videre blir brukt til å utføre handlinger oppført i filen *YFY.cs* via kallet på metoden *readstuff()* med *path* variabelen som argument.
3. **readstuff(string place)** - Denne metoden starter med to *if statements* som validerer filens innhold og lokasjon. Om disse feiler vil det bli kastet en feilmelding til bruker og programmet vil avslutte.
Videre blir det deklart et objekt av *library* klassen med instans variabelen *l*". Det er denne variabelen som brukes til å lagre data stringer som blir produsert i denne metoden, og logikken for hvordan dette lagres blir definert i klassene *library* og *file* som finnes i filen *Litt_kode.cs*.
4. **using (var sr = new StreamReader(place))** - Dette er løsningen som leser selve filen og er siste ledd i metoden *readstuff(string place)*.
Her starter koden med å åpne filen ved bruk av *Streamreader*.
Etter dette er det en *while* loop som deler opp teksten i stringer ved bruk av logikken som blir definert i metoden *splitstuff()*.
Når hele filen er blitt iterert gjennom, returneres resultatet til *library* objektet som er blitt deklart som variabelen *l*".
5. **Utskrift** - Når *main* metoden mottar generert data, starter den en *for loop* som teller antallet linjer i *library* objektet og itererer gjennom liste objektet.
For hver iterasjon printes linjenummer med tilhørende string ut til konsollen frem til listen er tom.
6. **Programmet avsluttes** - Når iterasjonen er fullført har programmet kjørt ferdig og avsluttes som forventet.

1.1.5 Endring på uttrykk i kode

I denne seksjonen er det blitt jobbet noe i gruppe, men siden mye av tiden i workshop gikk til forståelse av kodestruktur så er en god del av disse forslagene til endring blitt utført post-workshop. I perioden under selvstendig arbeid på dette er fagbok og ChatGPT blitt brukt som hjelpemidler.

Filnavn

Vedrørende navnkonsvensjoner for filnavn, så var det lite konkretisert i fagboken. Her har det blitt tatt i bruk *ChatGPT* for avklaring på navnkonsvensjoner vedrørende god konstruksjon av rammeverk.

Retningslinjer:

- Filnavn burde passe med hovedklassen i en fil.
- Filnavn skal følge PascalCasing.
- Det skal ikke brukes andre tegn enn *alfanumeriskere* karakterer.

Basert på dette ville det da være logisk å gi filnavn mer beskrivende navn.
Her er enkelte forslag på potensielle navn som ville være mer passende i denne konteksten.

- Litt_kode.cs → FileParts.cs
- YEY!.cs → FileProcessing.cs
- prog.cs → Program.cs

1.1.6 Klasser

Klasser faller under mange av de samme konvensjonene som gjelder filer, med noen ekstra retningslinjer.

Retningslinjer:

- Skal skrives i PascalCasing.
- Skal navngis som substantiv.
- Det skal ikke brukes "Ungarsknotasjon.
- Sammensatte ord bruker ikke PascalCasing.
- Klasser burde være beskrivende, slik at det er enkelt å forstå hensikten med den.

Basert på logikken vi har for fil regler, så vil det være logisk å navngi hovedklasser etter filnavn.
Dette er listen over forslag til endring av klassenavn.

- file → FileParts
- library → FileLibrary
- app → FileProcessing

1.1.7 Metoder

Metoder er kode som representerer en form for handling eller aktivitet. De har blant annet følgende retningslinjer.

Retningslinjer:

- Skal formuleres som verb.
- Skal følge PascalCasing.
- Skal være beskrivende i forhold til sin funksjon.
- Leselighet burde prioriteres over teknisk korrekthet.

Basert på følgende retningslinjer, foreslås følgende endringer av metode navn.

- readstuff → ReadFile
- splitstuff → SplitText
- ToStringO → ToString

1.1.8 Variabler

Det finnes svært mange diskusjoner vedrørende hva som er riktig fremgang når man skal opprette nye variabler. Men i denne rapporten, så prøves det å forholde seg til fagbok sine retningslinjer til beste evne. Her står det blant annet at camelCasing er forbeholdt parametere, så selv om det finnes argumenter for at camelCasing også skal brukes på lokale variabler i private klasser, så ser vi bort i fra dette for denne gang. På grunnlag av dette, så foreligger det blant annet følgende retningslinjer.

Retningslinjer:

- Skal skrives med PascalCasing.
- Skal beskrive sin hensikt.
- Navnform skal være substantiv eller substantiv frase.

På grunnlag av dette foreligger følgende forslag til endring.

- I → FileLibrary
- place → FilePath
- current → CurrentLine
- chunk → TextLine

- result → TextParts
- looking → insideQuotes

1.1.9 Konstanter

I denne filen er det ikke blitt deklarert noen konstanter, men dette burde vært vurdert for separatoren som brukes til å skille ut tekststrenger i metoden *splitstuff(string chunk)*. Dette er et forslag på hvordan dette vil gjort koden mer lesbar og lettere å endre ved et senere tilfelle.

Kode 1.1: Forslag til endring av metode ved bruk av konstanter

```
private const char QuoteCharacter = '"';
private const char CharacterDelimiter = ',';

private List<string> splitstuff(string chunk)
{
    var result = new List<string>();
    bool looking = false;
    var slice = new StringBuilder();

    for (int index = 0; index < chunk.Length; index++)
    {
        char item = chunk[index];

        if (item == QuoteCharacter)
        {
            if (looking && index + 1 < chunk.Length && chunk[index
                + 1] == QuoteCharacter)
            {
                slice.Append(QuoteCharacter);
                index++;
            }
            else
            {
                looking = !looking;
            }
        }
        else if (item == CharacterDelimiter && !looking)
        {
            result.Add(slice.ToString());
            slice.Clear();
        }
        else
        {
            slice.Append(item);
        }
    }
    result.Add(slice.ToString());
    return result;
}
```

1.2 Del 2

1.2.1 Gruppen vi møtte

Etter at vi var ferdige med første del av workshop gikk vi tilbake til klasserommet for å møte gruppen som vi skulle rapportere til. Vi møtte følgende gruppe, som bestod av disse deltagerne.

Gruppenummer:

- Gruppe 15 (supergruppe 7)

Deltagere:

- Tord Vestby
- Aram Naji Thalje
- Ibaida Taha

1.2.2 Referat

Etter hver gruppe hadde lagt frem sine funn og potensielle endringer, så vi at vi hadde kommet frem til relativt de samme konklusjonene, til tross for at vi hadde formulert oss noe forskjellig. Gruppen vi møtte hadde kommet en del lengre enn oss når det kom til å kontrollere semantiske og syntaktiske feil i koden, så vi fikk noen gode tips til hvor burde starte å se på ting. På samme tid så hadde vi gjort enkelte oppdagelser som den andre gruppen hadde oversett.

Konklusjonen vår etter at gruppene hadde møtes, var at det alltid vil være behov for kontinuerlig retting av større prosjekter med bruk av flere rettere. Da det en overser vil kanskje en annen oppdage. Vi fikk også gjennom denne eksempelkoden en erfaring på hvor viktig det er med disse følgereglene, da det ville vært et sant mareritt å overta et større prosjekt hvor all (fungerende) kode hadde vært skrevet på tilsvarende måte.

Workshop 2

2.1 Gruppen vår

Etter at vi hadde blitt introdusert for oppgaven, ble vi delt inn i tilfeldige grupper til å utføre workshop. Min gruppe ble tildelt følgende gruppe nummer og bestod av disse deltagerne.

Gruppenummer:

- 2

Deltagere:

- Andreas Isaksen
- Andreas Olaussen
- Abdelkader Adam Niane
- Erik Hankø

2.2 Analyse av koden

Vi fordelte de to bibliotekene mellom gruppen, og deretter jobbet i lag på to, hvor vi videre lagde oss noen flertall hypotetiske scenario, med tilhørende pseudo-kode. Etter dette prøvde vi å gå gjennom utlevert bibliotek for å se etter semantiske feil og dårlige navnkonvensjoner.

2.2.1 Library 1 (Andreas Isaksen, Andreas Olaussen)

2.2.1.1 Scenario 1 – Prisjakt komplett PC (Prissammenligning)

- Kort om: En webløsning som henter produkter med spesifikasjoner for en komplett PC og gir brukeren alternativer basert på oppført budsjett. Denne løsningen vil ikke gi muligheter til å utføre en komplett bestilling, men heller gi brukeren en oversikt over hvilke komponenter som er tilgjengelige og hvilke butikker, samt en prissammenligning.

Kode 2.1: Pseudo kode for Scenario 1

```
static void Main(string[] args)
{
    // Definerer komponenter som skal sammenlignes
    List<PcComponent> componentsToCompare = new List<
    PcComponent>
```

```

{
    new PcComponent { Name = "Intel_i7_12700K", Price = 0
    },
    new PcComponent { Name = "Corsair_Vengeance_16GB_RAM",
    Price = 0 }
};

// Definerer tilgjengelige butikker
List<IShop> shops = new List<IShop>
{
    new Shop("ShopA", 500m, 50m),
    new Shop("ShopB", 750m, 70m),
    new Shop("ShopC", 1000m, 0m)
};

// Sammenligner priser fra de forskjellige leverandørene
foreach (var component in componentsToCompare)
{
    Console.WriteLine($"Comparing_prices_for:{component.
    Name}");
    foreach (var shop in shops)
    {
        decimal price = shop.GetComponentPrice(component);
        Console.WriteLine($"Shop:{shop.ShopName},Price:{
        price},ShippingCost:{shop.ShippingCost}");
    }
}
}

```

2.2.1.2 Scenario 2 – Lagerstyring for PC-leverandør

- Kort om: Dette kan være et potensielt lagerstyringssystem for en PC-leverandør til større kunder.

Kode 2.2: Pseudo kode for Scenario 2

```

static void Main(string[] args)
{
    // Lager en liste over bestillinger
    List<ShopOrder> orders = new List<ShopOrder>();

    // Definerer en butikk med inventar
    IShop shop = new Shop("Atea", 800m, 120m);
    shop.AddInventory(new List<PcComponent>
    {
        new PcComponent { Name = "NVIDIA RTX 4090", Price =
        16000m },
        new PcComponent { Name = "AMD Ryzen 9 5900X", Price =
        5000m }
    });

    // Kunde lager en bestilling
    ShopOrder customerOrder = new ShopOrder

```

```

{
    Shop = shop,
    OrderedComponents = new List<PcComponent>
    {
        new PcComponent { Name = "NVIDIA_RTX_4090", Price =
            16000m },
        new PcComponent { Name = "AMD_Ryzen_9_5900X", Price
            = 5000m }
    }
};

// Legger til ordre i butikkens system
orders.Add(customerOrder);

// Regner ut total kostnad for kunde
decimal totalCost = customerOrder.TotalCost + shop.
    CalculateShippingCost(customerOrder.TotalCost);
Console.WriteLine($"Total_Order_Cost_(including_shipping):_
    {totalCost}");
}

```

2.2.1.3 Scenario 3 – API løsning som bygger en PC basert på valgte komponenter innenfor et budsjett

Kode 2.3: Pseudo kode for Scenario 3

```

. // Definere Budsjett
Budget userBudget = new Budget { MaxAmount = 15000m };

// Liste med komponenter for å bygge PC
List<PcComponent> requiredComponents = new List<PcComponent>
{
    new PcComponent { Name = "GPU", Price = 0 },
    new PcComponent { Name = "CPU", Price = 0 },
    new PcComponent { Name = "RAM", Price = 0 },
};

// Definerer ekvivalente grupper som ulike modeller
List<IEquivalentGroup> equivalentGroups = new List<
    IEquivalentGroup>
{
    new EquivalentGroup
    {
        Components = new List<PcComponent>
        {
            new PcComponent { Name = "RTX_3060", Price = 4000m
            },
            new PcComponent { Name = "RX_6700_XT", Price = 3800
            m }
        }
    }
}

```

```
};

// Definere ulike butikker med forskjellige priser
List<IShop> availableShops = new List<IShop>
{
    new Shop("Elkjøp", 500m, 100m),
    new Shop("Power", 1000m, 150m),
};

// Bruke OrderOptimizer for å finne beste alternativ
IOrderOptimizer optimizer = new OrderOptimizer();
OptimizationResult result = optimizer.OptimizeOrders(
    requiredComponents, equivalentGroups, availableShops,
    userBudget
);

// Vise resultater med en metode
ShowResults(result);
```

2.2.1.4 Library 1 – Retningslinjer som biblioteket bryter

Vi brukte ganske mye tid på scenario og pseudo kode på denne oppgaven. Dermed rakk vi dessverre ikke å gå veldig dypt i å se etter feil i utlevert bibliotekskode.

Vi gjorde følgende funn:

- PcComponent og ShopOrder
Dette er structs med mutable egenskaper. Structs skal være immutable. De burde heller vært klasser.
- Public bool IsWithingBuget{get; set;}
Dette er en bool metode og skal derfor ikke har get/set implementasjon.

2.2.2 Library 2 (Erik Hankø, Abdelkader Adam Niane)

2.2.2.1 Scenario 1 – Bygg en PC med begrenset budsjett

- Budsjett: 10000 kr
- Nødvendige komponenter: Processor, Memory, GraphicsCard, Storage, Motherboard, Power Supply, Case

Kode 2.4: Scenario 1: Bygg en PC med begrenset budsjett

```
PCBuildConfig config = new PCBuildConfig
config.Budget = 10000
config.RequiredCategories = [Processor, Memory, GraphicsCard,
    Storage, Motherboard, Power Supply, Case]
```

```

stores = List<Store>
// Populate stores with store data

DefaultOrderOptimizer optimizer = new DefaultOrderOptimizer()
orderStrategy = optimizer.OptimizeOrders(config, stores)

print("Total_Cost:_ " + orderStrategy.TotalCost)
for storeOrder in orderStrategy.StoreOrders:
    print("Store:_ " + storeOrder.StoreName)
    for product in storeOrder.Products:
        print("-_ " + product.Name + ":_ " + product.Price)
    print("Shipping_Cost:_ " + calculateShippingCost(storeOrder)
        ) // Calculate if necessary

```

2.2.2.2 Scenario 2 – Optimalisere ordre med flere butikker

- Ulike butikker som tilbyr forskjellige priser og fraktkostnader.
- Målet er å finne den kombinasjonen av butikker som gir den laveste totale kostnaden, inkludert frakt.

Kode 2.5: Scenario 2: Optimalisere ordre med flere butikker

```

// Assume similar config and stores initialization as Scenario
1

DefaultOrderOptimizer optimizer = new DefaultOrderOptimizer()
orderStrategy = optimizer.OptimizeOrders(config, stores)

print("Total_Cost:_ " + orderStrategy.TotalCost)
for storeOrder in orderStrategy.StoreOrders:
    print("Store:_ " + storeOrder.StoreName)
    for product in storeOrder.Products:
        print("-_ " + product.Name + ":_ " + product.Price)
    print("Shipping_Cost:_ " + storeOrder.ShippingCost)

```

2.2.2.3 Scenario 3 – Håndtering av ekvivalensgrupper

- Noen produkter anses som ekvivalente (f.eks., forskjellige merker av DDR4 RAM).
- Målet er å gruppere produktene og velge det billigste alternativet innen hver gruppe.

Kode 2.6: Scenario 3: Håndtering av ekvivalensgrupper

```

products = List<Product>
// Populate products with product data including
    EquivalentGroup

groupedProducts = EquivalenceMatcher.GroupByEquivalence(
    products)

```

```
for groupName, groupProducts in groupedProducts:
    print("Group:␣" + groupName)
    for product in groupProducts:
        print("-␣" + product.Name + ":␣" + product.Price)
```

2.2.2.4 Library 2 – Retningslinjer som biblioteket bryter

- **Navngivning og konvensjoner:**

Kommentarer bør ikke ha dobbel skråstrek (///) hvis biblioteket bruker en annen stil for dokumentasjon.

- **Strukturell oppdeling:**

Enklere strukturer eller klasser bør vurderes for delene som har enkel oppførsel (for eksempel ShippingCost kan være en egen klasse eller struktur om den skal inneholde mer logikk senere).

- **Bruk av public setter:**

publicsettere bør vurderes om de burde vært immutable (readonly) for bedre innkapsling. Dette kan unngå utilsiktede endringer av eksterne koder.

- **Organisering og modularisering:**

Håndtering av ekvivalensgrupper kan integreres bedre i selve optimization-prosessen, i stedet for å være en separat statisk klasse.

- **Vurdering av bibliotekets struktur:**

Generelt ser biblioteket ut til å være ganske godt strukturert, men det er noen forbedringsmuligheter:

- Bedre navngivning og konsistens: Sørge for at alle navn følger samme konvensjon og er selvforklarende.
 - Innkapsling: Vurder bruk av private settere og readonly hvor det passer.
 - Bruk av interfacer: Det er bra at biblioteket bruker interfacer for optimizer, dette gjør den utbyttbar og testbar.
 - Ekstra validasjon og feilhåndtering: Vurder å inkludere mer robust feilhåndtering og validasjon, spesielt rundt økonomiske data som budsjett og priser.
- En mulig bedre løsning kunne være å:
 - Inkludere flere detaljer i Optimizer som kan ta hensyn til spesifikke kundepreferanser, som preferanser for merker eller spesifikke modeller.
 - Introduksjon av mer fleksible regler for filering og evaluering av produkter, for eksempel ved å bruke strategi- eller regelmønstre.
 - Til slutt må vi også teste biblioteket med faktiske data for å se hvor godt det fungerer i praksis.
-

2.3 Referat

Etter at vi hadde fullført alle overordnede oppgaver, hadde igjen 10 minutter til å sammenligne funn og diskutere koden.

Det ble i hovedsak å lese opp funn for og pseudo struktur.

Etter dette konkluderte vi med at det var gjort gode funn og funnet interessante, samt relativt like løsninger på hva bibliotekene kunne brukes til.

Workshop 3

3.1 Gruppen vår

Etter at vi hadde blitt introdusert for oppgaven, ble vi delt inn i tilfeldige grupper til å utføre workshop. Min gruppe ble tildelt følgende gruppe nummer og bestod av disse deltagerne.

Gruppenummer:

- 10

Deltagere:

- Andreas Isaksen
- Emil Berglund
- Klaus Haugnæss
- Ida Tollaksen
- Hedda Ørnelund Nilsen

3.2 Organisering

Etter at alle medlemmer var samlet og passende arbeidssted var blitt allokert, startet vi med videre oppdeling av gruppen.

Grunnet at vi var en gruppe på fem, valgte vi å dele opp gruppen slik at den gruppen med flest medlemmer håndterte oppgaven med lengst kode.

Gruppefordeling

S1:

- Andreas Isaksen
- Emil Berglund

S2:

- Klaus Haugnæss
- Ida Tollaksen
- Hedda Ørnelund Nilsen

Gruppe S1 fikk rolle som utviklere for utdelt kode tilhørende 'Forelesning 4 - Workshop pakke for S1', og gruppe S2 fikk rolle som utviklere for utdelt kode tilhørende 'Forelesning 4 - Workshop pakke for S2'.

3.3 Analyse av koden

Begge grupper startet separat med å bygge et applikasjons prosjekt basert på utdelt kode.

Gruppe S1 brukte *Visual Studio* til å bygge prosjektet.

Gruppe S2 brukte *Visual Studio Code* til å bygge prosjektet.

Etter at prosjektene var satt opp, brukte hver gruppe litt tid på å sette seg inn i strukturen til sine representative klasser.

Vi kunne konkludere med at begge løsninger omhandlet behandling av tekststrenger, men med differensierende hensikter.

Løsningen til gruppe S1 omhandlet i hovedsak om *parsing* av data fra en '.csv' fil.

Løsningen til gruppe S2 omhandlet søke- og erstatningsfunksjonalitet av data fra en '.txt' fil.

Da oppgaven insinuerte at det ikke burde brukes langt tid på dette, valgte vi å gå videre til neste oppgave.

I en profesjonell situasjon ville det være hensiktsmessig å bruke mer tid på dette steget, slik at man har en bedre innsikt i løsningen man ønsker å teste på brukere.

3.4 Brukertest

Det ble tatt en avgjørelse om at gruppe S1 skulle innta rollen som utviklere i første runde. Etter at første brukertest var utført, skulle vi snu på rollene.

3.4.1 S1-medlemmer observerer S2-medlemmer

3.4.1.1 Oppgave

Medlemmene i S2 fikk S1-sitt prosjekt, ferdig oppsatt, med en tom main-funksjon.

S2 fikk følgende oppgavetekst som de skulle bruke til å utføre oppgaven:

Oppgave A: Bruk første overload i `DataParser.cs`

Scenario: Du har en CSV-fil med navnet `data.csv` i samme mappe som koden din. Du ønsker å parse denne og skrive ut den første raden med faktisk data.

Metoden du skal bruke:

`ParseCsv(string filePath, bool hasHeader, char delimiter = ',')`

Oppgave B: Bruk andre overload i `DataParser.cs`

Scenario: Du skal parse samme fil, men denne gangen må du bruke `FileInfo` og et `ParserOptions`-objekt.

Metoden du skal bruke:

ParseCsv(FileInfo file, ParserOptions options)

3.4.1.2 Rollefordeling

Fasilitator

- Andreas Isaksen

Observatør og skribent

- Emil Berglund

Brukertester

- Klaus Haugnæss
- Ida Tollaksen
- Hedda Ørnelund Nilsen

3.4.1.3 Fremgangsmåte – Oppgave A

- 1) Medlemmene fant CSV fil i Downloads lokalt, og flyttet denne inn i prosjekt mappen.
- 2) Medlemmene begynte med å implementere metoden ParseCsv.
 - Fant filsti.
 - Fant ut av at filen har header og satt bool verdi til true.
 - Slet litt med å finne ut av tredje parameter.
 - Fikk veiledning om at dette skulle være en char og være «omringet» av fnutter.
 - Fikk build error ved kjøring av programmet.
 - Fikk info om at det trengs et objekt og en liste for at metoden ParseCsv skal fungere.
 - Fant ut av at de trengte en liste med string som value.
 - Lagde instanse av DataParser.
 - Fikk Error: Wrong Filepath.
 - Ettersom de fikk feil path, prøvde de absolute-path.
 - Programmet kjører, men ingenting skrives ut.
 - Fant ut at programmet returnerer noe, og printes derfor ikke.
 - Medlemmene benytter Console.WriteLine og ble tipset om string.Join.
 - Får skrevet ut objektid, og ingen tekst.
 - Ble tipset om at det skal skrives ut ved å kalle på liteobjektet, og ikke en variabel av DataParser.
 - Ved å gjøre dette, og benytte string.Join med seperator, og skrive ut listeobjektet med bestemt index, fikk de ut ønsket resultat.

3.4.1.4 Fremgangsmåte – Oppgave B

1) Arbeidet videre fra Oppgave A

- Medlemmene lagde to objekter, FileInfo og ParserOptions.
- Videre lagde de et objekt av ParseCsv med disse to objektene som parametere.
- Videre var fremgangsmåten mye likt som Oppgave A, med nye parametere. Ønsket resultat ble deretter skrevet ut.

3.4.1.5 Program kode skrevet av gruppe S2

Kode 3.1: S2 – Oppgave A && B

```
using Workshop.Data;

namespace Workshop.CsvParseCLI
{
    internal class Program
    {
        static void Main(string[] args)
        {
            //Oppgave A
            var dataParser = new DataParser();

            var temp1 = dataParser.ParseCsv("C:\\Users\\andyi\\Dropbox\\IT_Skole\\hiof\\.Net\\workshop\\workshop3\\Workshop\\infile\\names.csv", true, ',');

            Console.WriteLine(String.Join(',', temp1[0]));

            //Oppgave B
            var filePath = new FileInfo("C:\\Users\\andyi\\Dropbox\\IT_Skole\\hiof\\.Net\\workshop\\workshop3\\Workshop\\infile\\names.csv");

            var parser_options = new ParserOptions
            {
                HasHeader = true
            };

            var temp2 = dataParser.ParseCsv(filePath, parser_options);

            Console.WriteLine(String.Join(',', temp2[0]));
        }
    }
}
```

3.4.2 S2-medlemmer observerer S1-medlemmer

3.4.2.1 Oppgave

Medlemmene i S1 fikk S2-sitt prosjekt, ferdig oppsatt, med en tom main-funksjon. S1 fikk følgende oppgavetekst som de skulle bruke til å utføre oppgaven:

Oppgave A: Bruke metode med flere parametere

Scenario: Du har en tekstfil, for eksempel example.txt, som inneholder noen forekomster av ordet "Hello" i ulike varianter (store/små bokstaver). Du ønsker å erstatte "Hello" med "Hi" og vurdere om du samtidig vil være case-sensitiv (f.eks. "Hello" vs. "hello"). Du vil også kanskje bestemme deg for om du vil opprette en sikkerhetskopi (.bak) av originalfilen.

Metoden du skal bruke:

```
int SearchAndReplace(  
    string filePath,  
    string searchText,  
    string replacementText,  
    bool caseSensitive,  
    bool createBackup = false  
);
```

Oppgave B: Bruke et SearchReplaceOptions-objekt

Scenario: Denne gangen skal du gjøre en tilsvarende eller litt annerledes tekstsøking og -erstatting, men ved å spesifisere alt i et konfigurasjonsobjekt i stedet for i parametere.

Metoden du skal bruke:

```
int SearchAndReplace(SearchReplaceOptions options);
```

3.4.2.2 Rollefordeling

Fasilitator

- Klaus Haugnæss

Observatør og skribent

- Ida Tollaksen
- Hedda Ørnelund Nilsen

Brukertester

- Andreas Isaksen
- Emil Berglund

3.4.2.3 Fremgangsmåte – Oppgave A

1. Medlemmene gikk igjennom mappestruktur og innhold.
2. Medlemmene tittet etter hvor metoden SearchAndReplace ligger, de fant den i FileTextReplacer.cs
3. De gjør seg kjent med hvilken parameter metoden bruker. De sier høyt at det er en konstruktør, opprette søketekst, file, true false, casesensitiv, definere backup.
4. Medleme opprettet et FileTextReplacer objekt i main, prøver først å sette parameterene inne i oppretelsen av objektet.
5. Fikk hint om at man kanskje burde sette parameterene utenfor oppretelsen.
6. Videre prøvde de å undersøke hvordan de kunne gjøre dette. Får file not found error til slutt.
7. Forsøker å ordne filstid ved å bruke absolutt filstiden, det fungerer, backup ble opprettet

3.4.2.4 Fremgangsmåte – Oppgave B

Grunnet manglede tid fikk ikke gruppe S1 mulighet til å fullføre oppgave B.

3.4.2.5 Program kode skrevet av gruppe S1

Kode 3.2: S1 – Oppgave A

```
public static void Main(){
    FileTextReplacer searchndreplace = new
        FileTextReplacer( ) :

        int potions = searchndreplace.
            SearchAndReplace(
                @"C:\Users\idak\OneDrive_\Østfold_
                University
College\Dokumenter\Høgskole\Var25\.Net\Workshops\Workshop3\
                Workshop3\fil.txt",
                "Hello",
                "Hi",
                caseSensitive: true,
                true
            );
    Console.WriteLine($"Number_of_replacements:
        _{options}");
}
```

3.5 Referat

Grunnet manglede tid på slutten av prosjektet, fikk ikke gruppen vår mulighet til å utføre en dialog rundt kodens struktur, samt de potensielle funnene som ble gjort.

På grunnlag av dette har det blitt gjort en selvstendig vurdering på dette, som kommer frem i oppsummeringen.

3.6 Oppsummering

Denne oppsummeringen er utført på eget initiativ etter at dagens eksperiment var avsluttet, og burde dermed ikke bli anset som en helhetlig oppsummering fra gruppen, da dette ikke er bekreftet som en union oppfatning fra gruppen som en helhet. Men heller mine egne personlige synspunkter og vurderinger.

Til tross for at dette var et veldig interessant eksperiment, så forfalt en del av den potensielle kvaliteten på kode- analyse og skriving grunnet kort behandlingstid.

Det mistenkes at det ville være mer intuitivt å ta i bruk diverse metode kall om gruppen hadde fått lengre tid til å sette seg inn i strukturens logikk.

3.6.1 Oppgave 1 - Kode

3.6.1.1 Oppgave A

Vedrørende implementasjon av selve metoden i seg selv var ganske grei og rett frem.

Det oppstod derimot en del forvirring angående de data som metoden returnerte etter kallet.

Dette kan være grunnet noe manglende kompetanse når det kommer til logisk oppbygging av fundamentale klasse objekter. Men vi hadde utfordringer med å få ut de ønskede data fra det returnerte objektet.

Til slutt ble det nødvendig for oss å bruke metode kallet Join() fra String klassen , for å klare å hente ut de korrekte linjene.

Dette er en logikk som mulig burde vært implementert i metode logikken, slik at det ikke ville vært nødvendig i selve Program klassen.

Et mulig alternativ ville vært å laget en ny metode som utførte denne logikken basert på objektet som blir returnert i den overordnede metoden, med parametere som definerte hvilke data/liner som var ønskelige å få ut.

3.6.1.2 Oppgave B

Denne løsningen var nok også litt utfordrende grunnet manglende fundamental kompetanse vedrørende de offentlige klassene(slik som FileInfo klassen).

Her var det også relativt enkelt å kalle på metoden etter at vi hadde lest en del om overordnede klasser som blir brukt i metoden og hvordan vi kunne opprette de.

Har hadde vi lært av Oppgave 1, så det å få ønsket resultat her var mye lettere. Men slik som om oppgave 1, burde det også mulig være en metode som utfører det siste logiske steget for å få ut ønskede data.

3.6.2 Oppgave 2 - Kode

3.6.2.1 Oppgave A

I denne oppgaven merket vi at det ikke var veldig intuitivt hvordan selve konstruktøren til løsningen var bygget opp. Det måtte flere forsøk til før riktig datatype ble tatt i bruk for å kunne opprette objektet.

Når dette var løst var resterende arbeid for å få løsningen til å fungere relativt greit.

Her ville jeg kanskje ha tatt med noe mer dokumentasjons for hvordan opprette objekter, men dette er noe som måtte bli tatt opp til vurdering, basert på hvilken grad av kode erfaring den tenkte bruker har. Her igjen ville det nok godt vesentlig mye bedre fra starten med større tidsrammer for å sette seg inn i løsningen.

3.6.2.2 Oppgave B

Denne oppgaven ble ikke gjennomført og det er dermed ikke gjort noen videre vurdering på koden.